Language Technology

Chapter 3: Corpus Processing Tools

https://link.springer.com/chapter/10.1007/978-3-031-57549-5_3

Pierre Nugues

Pierre.Nugues@cs.lth.se

August 31 and September 3, 2026



Corpora

A corpus is a collection of texts (written or spoken) or speech recordings. Corpora are typically balanced across different sources: news, novels, etc.

	English	French	German
Most frequent words in a collection of contemporary running texts	*the*	*de*	*der*
	of	*le* (article)	*die*
	to	*la* (article)	*und*
	in	*et*	*in*
	and	*les*	*des*
Most frequent words in Genesis	*and*	*et*	*und*
	the	*de*	*die*
	of	*la*	*der*
	his	*à*	*da*
	he	*il*	*er*



Characteristics of Current Corpora

Large:

- The Bank of English (Collins and University of Birmingham) from the 1990s contained more than 500 million words.
- The Colossal Clean Crawled Corpus (2020) contains 156 billion English tokens.

Available in many languages.
Easy to collect: the web is the largest corpus ever built and is accessible with a single mouse click.
Parallel corpora contain the same text in two or more languages (e.g., English/French Canadian Hansards, European Parliament proceedings in 23 languages).
Annotated with part-of-speech tags or manually parsed (treebanks):

- Characteristics/N of/PREP Current/ADJ Corpora/N
- (NP (NP Characteristics) (PP of (NP Current Corpora)))



Lexicography

Writing dictionaries.
Dictionaries for language learners should be based on real usage:

- *They're just trying to score* **brownie points** *with politicians*
- *The boss is pleased — that's another* **brownie point**

Bank of English: *brownie point* (6 occurrences), *brownie points* (76 occurrences).
Extensive use of corpora to:

- Find **concordances** and cite real examples
- Extract **collocations** and describe frequent word pairs



Concordances

A word and its surrounding context:

Language	Concordances
English	s beginning of miracles did Je n they saw the miracles which n can do these miracles that t ain the second miracle that Je e they saw his miracles which
French	le premier des miracles que fi i dirent: Quel miracle nous mo om, voyant les miracles qu'il peut faire ces miracles que tu s ne voyez des miracles et des



Collocations

Word preferences: words that tend to occur together.

	English	**French**	**German**
You say	*Strong tea*	*Thé fort*	*Schmales Gesicht*
	Powerful computer	*Ordinateur puissant*	*Enge Kleidung*
You don't say	*Strong computer*	*Thé puissant*	*Schmale Kleidung*
	Powerful tea	*Ordinateur fort*	*Enges Gesicht*



Word Preferences

Strong *w*			Powerful *w*		
strong w	*powerful w*	*w*	*strong w*	*powerful w*	*w*
161	0	showing	1	32	than
175	2	support	1	32	figure
106	0	defense	3	31	minority
...					



Corpora as Knowledge Sources

Traditional use:

- Describe language usage more accurately

Machine learning:

- Learn statistical or machine-learning models for speech recognition, tagging, and parsing
- Evaluate tools such as part-of-speech taggers and parsers
- Derive models automatically from annotated or unannotated corpora

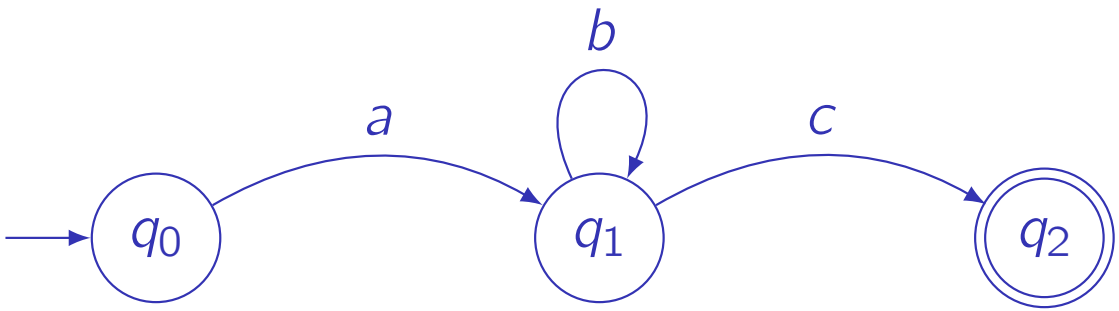
Applications:

- Machine translation
- Information and knowledge extraction
- Question answering from textual sources
- Dialogue systems



Finite-State Automata

A flexible tool for searching and processing text.
A finite-state automaton (FSA) accepts and generates strings such as *ac*, *abc*, *abbc*, *abbbc*, *abbbbbbbbbbbbc*, etc.





Finite-State Automata

Mathematically defined by:

- Q: a finite set of states
- Σ: a finite set of symbols or characters (the input alphabet)
- q_0: a start state
- F: a set of final states, $F \subseteq Q$
- δ: a transition function $Q \times \Sigma \rightarrow Q$, where $\delta(q, i)$ returns the state reached when the automaton is in state q and consumes the input symbol i



Regular Expressions

Regular expressions (regexes) are equivalent to finite-state automata and are generally easier to use.

1. Constant regular expressions:

Pattern	String
`regular`	*A section on regular expressions*
`the`	*The book of the life*

2. Metacharacters such as `*`. The automaton above corresponds to the regular expression `ab*c`.

```
$ grep 'ab*c' myFile1 myFile2
```

Although `grep` was the first widely used regex tool, most programming languages now follow the Perl syntax.



Main Regex Operations

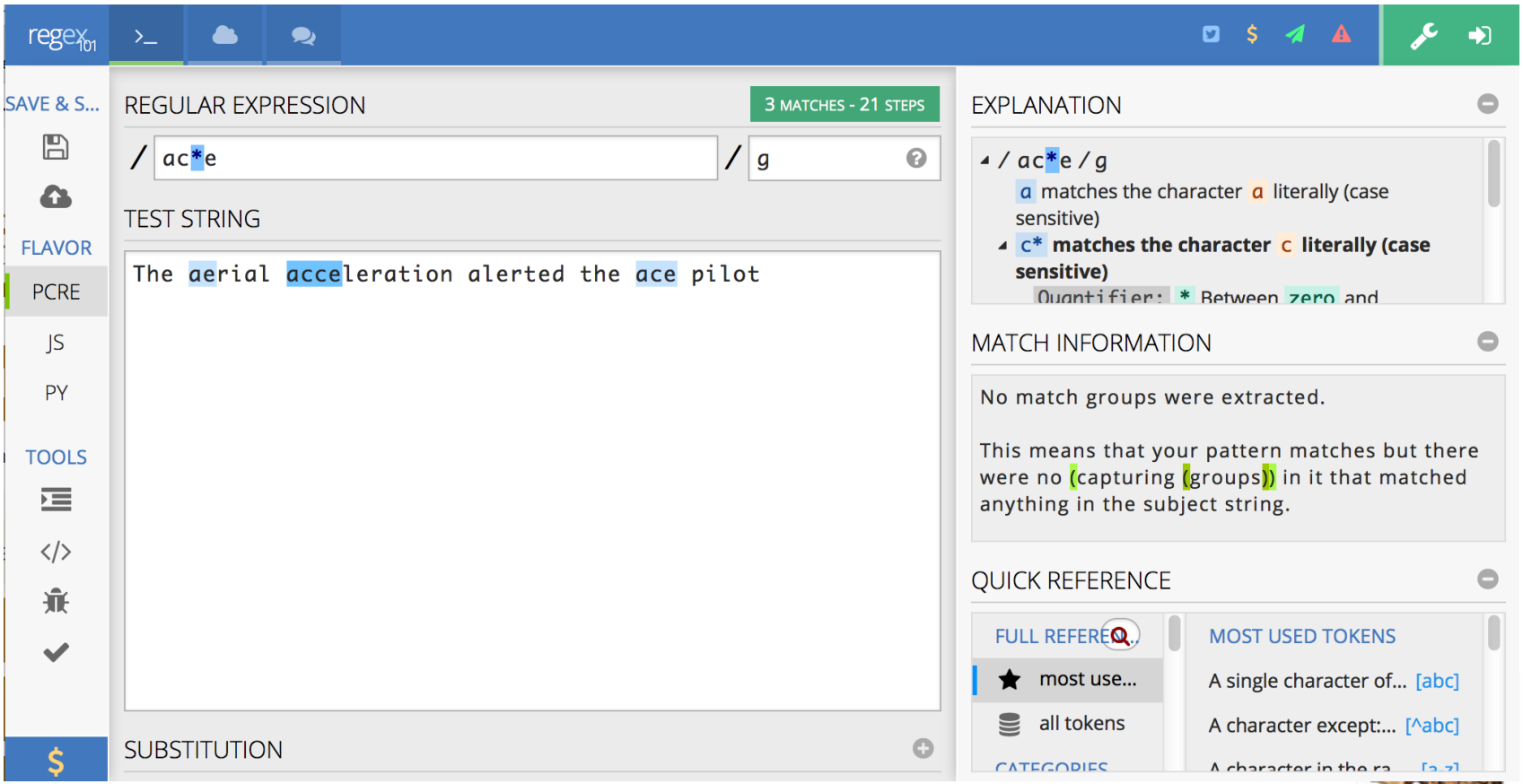
The two main regular-expression operations are matching and substitution.
They are often abbreviated using the Perl-style notation:

- The construct `m/regex/` denotes a match operation with the regular expression `regex`.
- The construct `s/regex/replacement/` performs a substitution: it matches the first occurrence of `regex` and replaces it with the `replacement` string.
- Modifiers can be added. For example, the `g` modifier (global) replaces all occurrences: `s/regex/replacement/g`.



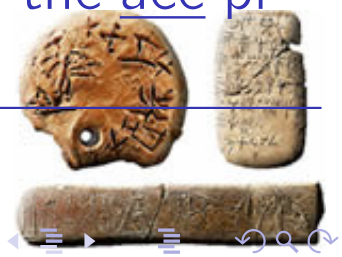
regex101.com

regex101.com: a useful website for experimenting with and testing regular expressions.



Metacharacters

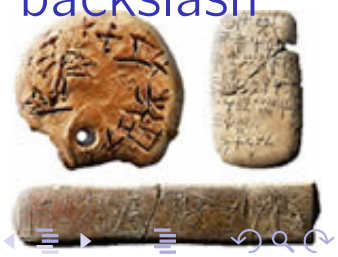
Chars	Description	Examples
*	Matches any number of occurrences of the preceding character (zero or more)	`ac*e` matches `ae`, `ace`, `acce`, `accce`, etc., as in "The aerial acceleration alerted the ace pilot"
?	Matches at most one occurrence of the preceding character (zero or one)	`ac?e` matches `ae` and `ace` as in "The aerial acceleration alerted the ace pilot"
+	Matches one or more occurrences of the preceding character	`ac+e` matches `ace`, `acce`, `accce`, etc., as in "The aerial acceleration alerted the ace pilot"



Metacharacters

Chars	Description	Examples
`{n}`	Matches exactly *n* occurrences of the preceding character	`ac{2}e` matches `acce` as in "The aerial acceleration alerted the ace pilot"
`{n,}`	Matches *n* or more occurrences of the preceding character	`ac{2,}e` matches `acce`, `accce`, etc.
`{n,m}`	Matches between *n* and *m* occurrences of the preceding character	`ac{2,4}e` matches `acce`, `accce`, and `acccce`

Literal occurrences of metacharacters must be escaped with a backslash (\).



The Dot Metacharacter

The dot `.` is a metacharacter that matches any single character except a newline.

`a.e` matches the strings *ale* and *ace* in:

> *The aerial acceleration alerted the ace pilot*

as well as *age*, *ape*, *are*, *ate*, *awe*, *axe*, or *aae*, *aAe*, *abe*, *aBe*, *a1e*, etc.

`.*` matches any string of characters up to a newline.



The Longest Match

The previous slide does not specify the matching strategy.
Consider the string *aabbc* and the regular expression `a+b*`.
By default the matching engine is greedy: it matches as early and as many characters as possible, resulting in `aabb`.
This can be problematic. Consider the regular expression `<b>.*</b>` and the phrase:

> *They match as early and as many characters as they can.*

A lazy strategy can be obtained with the `*?` quantifier: `<b>.*?</b>`, which yields:

> *They match as early and as many characters as they can.*



Character Classes

`[...]` matches any character contained in the list.
`[^...]` matches any character not contained in the list.
`[abc]` matches one occurrence of either `a`, `b`, or `c`.
`[^abc]` matches one occurrence of any character that is not `a`, `b`, or `c`.
`[ABCDEFGHIJKLMNOPQRSTUVWXYZ]` matches one uppercase unaccented letter.
`[0123456789]` matches one digit.
`[0123456789]+\.[0123456789]+` matches decimal numbers.
`[Cc]omputer [Ss]cience` matches `Computer Science`, `computer Science`, `Computer science`, and `computer science`.



Predefined Character Classes

Expr.	Description	Equivalent	Example
`\d`	Any digit	`[0-9]`	`A\dC` matches A0C, A1C, A2C, A3C, etc.
`\D`	Any nondigit	`[^0-9]`	
`\w`	Any letter, digit, or underscore	`[a-zA-Z0-9_]`	`1\w2` matches 1a2, 1A2, 1b2, 1B2, etc.
`\W`	Any nonword character	`[^\w]`	
`\s`	Any whitespace character	`[ \t\n\r\f]`	
`\S`	Any non-whitespace character	`[^\s]`	



Nonprintable Symbols or Positions

Char.	Description	Example
`^`	Matches the start of a line	`^ab*c` matches ac, abc, abbc, etc. at the beginning of a line
`$`	Matches the end of a line	`ab?c$` matches ac and abc at the end of a line
`\b`	Matches word boundaries	`\babc` matches `abcd` but not `dabc` `bcd\b` matches `abcd` but not `abcde`
`\n`	Matches a newline	`a\nb` matches a followed by a newline followed by b
`\t`	Matches a tab character	

```
egrep '^[aeiou]*$' myFile
```



Union and Boolean Operators

Union is denoted by |: `a|b` matches either `a` or `b`.
The expression `a|bc` matches the strings `a` and `bc`, while `(a|b)c` matches `ac` and `bc`.
Order of precedence:

1. Closure and other repetition operators (highest)
2. Concatenation, line and word boundaries
3. Union (lowest)

`abc*` denotes the set {`ab`, `abc`, `abcc`, `abccc`, ...}.
`(abc)*` denotes {`abc`, `abcabc`, `abcabcabc`, ...}.



Python

Match: m/regex/

```
import regex as re
line = 'The aerial acceleration alerted the ace pilot'
match = re.search('ab*c', line)
match # <regex.Match object; span=(11, 13), match='ac'>
match.group() # ac
```

The `re.search()` function stops at the first match.



Python

Use `findall()` or `finditer()` to obtain all matches.

Match: m/regex/g

```
match_list = re.findall('ab*c', line) # ['ac', 'ac']
```

Match: m/regex/g

```
match_iter = re.finditer('ab*c', line)
list(match_iter)
        # [<regex.Match object; span=(11, 13), match='ac'>,
        # <regex.Match object; span=(36, 38), match='ac'>]
```



Match Modifiers

Flags that modify the matching behavior.
These flags are equivalent to Perl's `m/regex/modifiers`.

- Case-insensitive: `i`. The instruction `m/regex/i`. In Python this corresponds to the flag `re.I`.
- Multiline: `m` (`re.M` in Python). `m/regex/m`.
- Single-line (dot matches newline): `s` (`re.S` in Python). With the `/s` modifier, a dot in `m/regex/s` matches any character including newlines.

In Python, flags are specified as a sequence separated by vertical bars (|).



Python

Match: m/regex/modifiers

```
text = sys.stdin.read()
match = re.search('^ab*c', text, re.I | re.M) # m/^ab*c/im
if match:
    print('-> ' + match.group())
```



Python

Substitute: s/regex/replacement/g

```
for line in sys.stdin:
    if re.search('ab+c', line):
        print("Old: " + line, end='')
        # Replaces all occurrences
        line = re.sub('ab+c', 'ABC', line) # s/ab+c/ABC/g
        print("New: " + line, end='')
```

Substitute: s/regex/replacement/

To replace only the first occurrence:

```
        # Replaces the first occurrence
        line = re.sub('ab+c', 'ABC', line, 1) # s/ab+c/ABC/
```



Backreferences

Parts of a matched pattern can be stored using backreferences. These parts are enclosed in parentheses in the regular expression, for example:

`^(.)(b+)c+`

- The first part, `(.)`, is stored in variable `\1`
- The second part, `(b+)`, is stored in `\2`, and so on.

Backreferences

```
line = 'abbbcdeeef'
match = re.search('^(.)(b+)c+', line)
match.group(1) # 'a'
match.group(2) # 'bbb'
```



Python

Backreferences

The pattern `m/(.)\1\1/` matches sequences of three identical characters:

```
line = 'abbbcdeeef'
match = re.search(r'(.)\1\1', line)
match.group(1) # 'b'
```

A raw string (prefix `r`) must be used; otherwise `\1` would be interpreted as an octal number.

Substitutions

```
s/(.)\1\1/***/g

re.sub(r'(.)\1\1', '***', 'abbbcdeeef') # 'a***cd***f'
```



Python

Multiple backreferences

Python can create as many capture groups as needed: \1, \2, \3, etc. Outside the regular expression these groups are accessed via group(<digit>): match_object.group(1), match_object.group(2), match_object.group(3), etc.

Multiple backreferences

```
m/\$ *([0-9]+)\.?([0-9]*)/

price = "We'll buy it for $72.40"
match = re.search(r'\$ *([0-9]+)\.?([0-9]*)', price)
match.group()  # '$72.40'  The entire match
match.group(1) # '72'      The first group
match.group(2) # '40'      The second group
```



Python

Substitutions

```
s/\$ *([0-9]+)\.?([0-9]*)/\1 dollars and \2 cents/g

price = "We'll buy it for $72.40"
re.sub(r'\$ *([0-9]+)\.?([0-9]*)',
       r'\1 dollars and \2 cents', price)
   # We'll buy it for 72 dollars and 40 cents
```



Python

Match objects

- `match_object.group()` or `match_object.group(0)` returns the entire match.
- `match_object.group(n)` returns the *n*-th parenthesized subgroup.

In addition, `match_object.groups()` returns a tuple containing all groups, and `match_object.string` contains the original input string.

```
price = "We'll buy it for $72.40"
match = re.search(r'\$ *([0-9]+)\.?([0-9]*)', price)
match.string   # We'll buy it for $72.40
match.groups() # ('72', '40')
```



Python

Match objects

The start and end indices of matched substrings can be obtained with:

```
match_object.start([group])
match_object.end([group])

line = """Tell me, O muse, of that ingenious hero
  who travelled far and wide after he had sacked
  the famous town of Troy."""
match = re.search(',.*,', line, re.S)
line[0:match.start()]                # 'Tell me'
line[match.start():match.end()] # ', O muse,'
line[match.end():]                   # 'of that ingenious hero
                                     # who travelled far and wide a
                                     # the famous town of Troy.'
```

A Regex to Find Concordances

To print concordances we need a regular expression that matches the pattern together with a left and right context.
For example, *Nils Holgersson* with a context of 15 characters:

```
.{0,15}Nils Holgersson.{0,15}
```

Ideally the pattern and the context width are passed as parameters:

```
pattern = 'Nils Holgersson'
width = 15
'.{0,width}pattern.{0,width}'
```



format()

`str.format()` provides variable substitution:

```
begin = 'my'
'{} string {}'.format(begin, 'is empty')
                                    # 'my string is empty'
```

`format()` offers many options, including argument reordering via indices:

```
begin = 'my'
'{1} string {0}'.format('is empty', begin)
                                    # 'my string is empty'
```

If the format string itself contains braces, they must be escaped by doubling them: `{{` for a literal `{` and `}}` for `}`.

```
('.{{0,{width}}}{pattern}.{{0,{width}}}'
 .format(pattern=pattern, width=width))
```



Concordances in Python

```
[file_name, pattern, width] = sys.argv[1:]
try:
    text = open(file_name).read()
except:
    print('Could not open file', file_name)
    exit(0)
# Spaces match tabs and newlines
pattern = re.sub(' ', r'\\s+', pattern)
# Replace newlines with spaces in the text
text = re.sub(r'\s+', ' ', text)
concordance = ('(.{{0,{width}}}{pattern}.{{0,{width}}})'
               .format(pattern=pattern, width=width))
for match in re.finditer(concordance, text):
    print(match.group(1))
```



Approximate String Matching

A set of edit operations that transform a source string into a target string: copy, substitution, insertion, deletion, and transposition (reversal). Edit examples for *acress* from Kernighan et al. (1990):

Typo	Correction	Source	Target	Position	Operation
acress	actress	–	t	2	Insertion
acress	cress	a	–	0	Deletion
acress	caress	ac	ca	0	Transposition
acress	access	r	c	2	Substitution
acress	across	e	o	3	Substitution
acress	acres	s	–	4	Deletion
acress	acres	s	–	5	Deletion



Building a Spell Checker

Spell checkers use a dictionary together with a set of transformations to suggest corrections for misspelled words.
Dictionaries are typically collected from well-written texts (novels, newspapers, etc.).

- Given a word that does not appear in the dictionary, the spell checker generates all single-edit transformations of that word.
- If only one edit operation is allowed on a source string of length n, and the alphabet consists of 26 unaccented letters:
  - deletion produces n new strings;
  - insertion produces $(n+1) \times 26$ strings;
  - substitution produces $n \times 25$ strings;
  - transposition produces $n-1$ new strings.
- The spell checker retains those transformations that exist in the dictionary and ranks them by frequency to suggest the most likely correction.



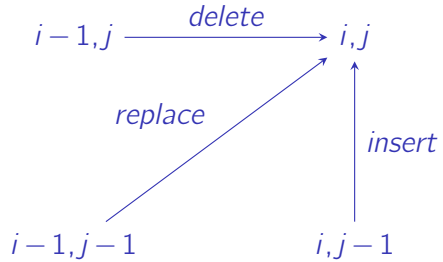
For an implementation see `http://norvig.com/spell-correct.html`.

Building a Spell Checker

```
freq('acres') = 36
freq('caress') = 3
freq('cress') = false
freq('actress') = 7
freq('access') = 56
freq('across') = 222
```



Distance between *ab* and *cb*



Edit distances measure the similarity between strings.

Consider the alignment

a	b	Source
c	b	Destination

b	2		
c	1		
Start	0	1	2
	Start	a	b



Minimum Edit Distance

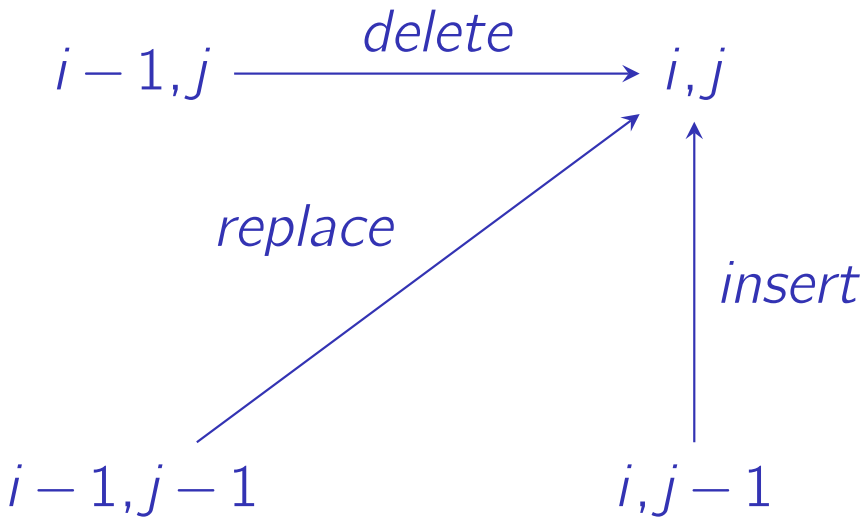
The minimum edit distance is computed using a matrix whose entry at position (i,j) is defined by the recurrence:

$$\text{edit_distance}(i,j) = \min \left\{ \begin{array}{l} \text{edit_distance}(i-1,j) + \mathit{del_cost} \\ \text{edit_distance}(i-1,j-1) + \mathit{subst_cost} \\ \text{edit_distance}(i,j-1) + \mathit{ins_cost} \end{array} \right\}$$

with base cases $\text{edit_distance}(i,0) = i$ and $\text{edit_distance}(0,j) = j$.



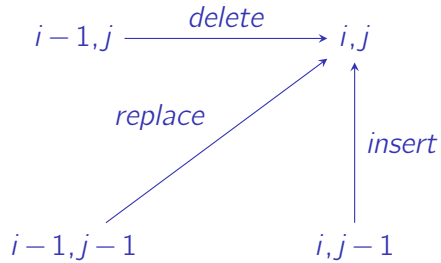
Edit Operations



Usually $del_cost = ins_cost = 1$,
$subst_cost = 2$ if $source(i) \neq target(j)$,
$subst_cost = 0$ if $source(i) = target(j)$.



Distance between *ab* and *cb*



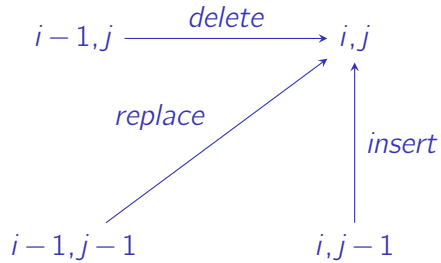
Alignment:

a	b	Source
c	b	Destination

b	2		
c	1	2	
Start	0	1	2
	Start	a	b



Distance between *ab* and *cb*



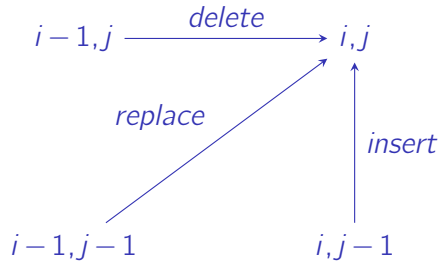
Alignment:

a	b	Source
c	b	Destination

b	2	3	
c	1	2	3
Start	0	1	2
	Start	a	b



Distance between *ab* and *cb*



Alignment:

a	b	Source
c	b	Destination

b	2	3	**2**
c	1	2	3
Start	0	1	2
	Start	a	b



Distance between *language* and *lineage*

e	7								
g	6								
a	5								
e	4								
n	3								
i	2								
l	1								
Start	0	1	2	3	4	5	6	7	8
	Start	l	a	n	g	u	a	g	e



Distance between *language* and *lineage*

e	7	6	5						
g	6	5	4						
a	5	4	3						
e	4	3	4						
n	3	2	3						
i	2	1	2	3	4	5	6	7	8
l	1	0	1	2	3	4	5	6	7
Start	0	1	2	3	4	5	6	7	8
	Start	l	a	n	g	u	a	g	e



Distance between *language* and *lineage*

e	7	6	5	6	5	6	7	6	**5**
g	6	5	4	5	4	5	6	5	6
a	5	4	3	4	5	6	5	6	7
e	4	3	4	3	4	5	6	7	6
n	3	2	3	2	3	4	5	6	7
i	2	1	2	3	4	5	6	7	8
l	1	0	1	2	3	4	5	6	7
Start	0	1	2	3	4	5	6	7	8
	Start	l	a	n	g	u	a	g	e



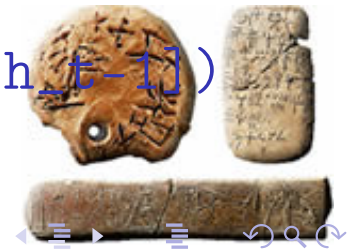
Python Code

```
[source, target] = sys.argv[1:]
length_s = len(source) + 1
length_t = len(target) + 1
# Initialize first row and column
table = [None] * length_s
for i in range(length_s):
    table[i] = [None] * length_t
    table[i][0] = i
for j in range(length_t):
    table[0][j] = j
```



Python Code

```
# Fill the table. Row and column indices start at 1
for i in range(1, length_s):
    for j in range(1, length_t):
        # Copy or substitution?
        cost = 0 if source[i-1] == target[j-1] else 2
        # Compute the minimum
        minimum = table[i-1][j-1] + cost
        if minimum > table[i][j-1] + 1:
            minimum = table[i][j-1] + 1
        if minimum > table[i-1][j] + 1:
            minimum = table[i-1][j] + 1
        table[i][j] = minimum
print('Minimum distance:', table[length_s-1][length_t-1])
```



Distance between *language* and *lineage*

	First alignment	Third alignment
Without epsilon symbols	l a n g *u* a g e | | | | / / / l i n e a g e	l a n g *u* a g e | | | / / / l i n e a g e
With epsilon symbols	l a n g u a g e | | | | | | | | l i n e ε a g e	l a n g u ε a g e | | | | | | | | l i n ε ε e a g e

